PROJETO ACADÊMICO EM PYTHON

Boletim Escolar Digital

Sistema interativo em modo terminal para gestão, acompanhamento acadêmico e persistência de dados.

Python 3.x Persistência em TXT CLI Interativo

PROPÓSITO DO SISTEMA

Objetivo e Problema Resolvido

O Desafio

Processos manuais de cálculo de notas estão sujeitos a falhas humanas, retrabalho e perda de registros quando a aplicação é encerrada.

A Solução

Uma ferramenta em Python segura, rápida e intuitiva que calcula médias automaticamente, valida entradas e armazena relatórios permanentes.

PILARES TÉCNICOS

Tecnologias e Conceitos Utilizados

Listas

Armazenamento ordenado de notas e coleção global de alunos.

Dicionários

Estruturação de dados no formato chave-valor (`nome` e `notas`).

Funções

Modularização do código para legibilidade e reuso de lógica.

Arquivos TXT

Persistência de dados permanente utilizando manipulação E/S (`open`).

MODELAGEM

Estrutura dos Dados em Memória

```
# Estrutura principal: Lista de Dicionários
alunos = [
  {
    "nome": "Maria Silva",
    "notas": [8.5, 7.0, 9.0]
  },
  {
    "nome": "João Santos",
    "notas": [5.0, 6.0, 4.5]
  }
]
```

Dicionário por Aluno

Organiza os dados individuais de maneira clara e extensível.

Lista Interna de Notas

Garante o armazenamento de exatamente 3 avaliações por estudante.

INTERAÇÃO

Funcionamento e Menu Interativo

```
def main():
  alunos = carregar_alunos()
  while True:
    print("\n═══ BOLETIM ESCOLAR DIGITAL ═══")
    print("1 - Cadastrar aluno")
    print("2 - Listar boletim")
    print("3 - Buscar aluno")
    print("4 - Estatísticas da turma")
    print("5 - Exportar relatório")
    print("0 - Sair")
```

Carregamento Automático

Recupera os alunos salvos no arquivo ao iniciar o programa.

Laço Contínuo (`while True`)

Mantém a execução até que a opção de saída (0) seja selecionada.

CADASTRO & TRATAMENTO DE ERROS

Cadastro e Validação de Notas

```python
def ler_nota(numero):
  while True:
    try:
      nota = float(input(f"Nota {numero} (0 a 10): "))
      if 0 ⩽ nota ⩽ 10:
        return nota
      print("Nota inválida! Digite entre 0 e 10.")
    except ValueError:
      print("Digite um número válido.")
```

Validação de Intervalo

Garante rigorosamente valores numéricos decimais entre 0.0 e 10.0.

Tratamento com `try/except`

Evita que o programa quebre se o usuário digitar letras ou símbolos.

REGRAS DE NEGÓCIO

Cálculo da Média e Classificação

```
def calcular_media(notas):
  return sum(notas) / 3

def situacao(media):
  if media ⩾ 7:
    return "Aprovado"
  elif media ⩾ 5:
    return "Recuperação"
  else:
    return "Reprovado"
```

Aprovado
Média maior ou igual a 7.0

Recuperação
Média entre 5.0 e 6.99

Reprovado
Média menor que 5.0

ARMAZENAMENTO PERMANENTE

Persistência no Arquivo `boletim.txt`

```
def salvar_alunos(alunos):
  with open(ARQUIVO, "w", encoding="utf-8") as arq:
    for aluno in alunos:
      linha = (
        f"{aluno['nome']};"
        f"{aluno['notas'][0]};"
        f"{aluno['notas'][1]};"
        f"{aluno['notas'][2]}\n"
      )
      arq.write(linha)
```

Formato Delimitado Por Ponto e Vírgula

Cada linha contém `Nome;Nota1;Nota2;Nota3` garantindo fácil leitura posterior.

Garantia de Manutenção de Dados

Os dados persistem intactos mesmo após fechar o programa terminal.

CONSULTAS

Busca e Exibição de Alunos

```
def buscar_aluno(alunos, nome):
  for aluno in alunos:
    if aluno["nome"].lower() == nome.strip().lower():
      return aluno
  return None
```

Busca Case-Insensitive

O método `.lower()` permite localizar estudantes independente de maiúsculas.

Formatador de Saída

A função `mostrar_aluno()` exibe nome, notas, média e status de forma limpa.

ANÁLISE DE DESEMPENHO

Estatísticas Globais da Turma

```python
# Média geral e Destaque
medias = [calcular_media(a["notas"]) for a in alunos]
media_turma = sum(medias) / len(medias)
maior_media = max(medias)
melhor_aluno = alunos[medias.index(maior_media)]
```

List Comprehension

Calcula rapidamente todas as médias da turma em uma linha elegante.

Contadores de Situação

Classifica e contabiliza a quantidade total de aprovados, em recuperação e reprovados.

EXPORTAÇÃO DE DADOS

Geração do Arquivo `relatorio.txt`

```
def exportar_relatorio(alunos):
  with open("relatorio.txt", "w", encoding="utf-8") as arq:
    for aluno in alunos:
      media = calcular_media(aluno["notas"])
      resultado = situacao(media)
      arq.write(f"{aluno['nome']};{media:.2f};{resultado}\n")
  print("Relatório exportado para relatorio.txt!")
```

Resumo Consolidado

Exporta a média formatada com duas casas decimais e o resultado final.

Pronto para Integração

Permite leitura simples em planilhas ou outros softwares educacionais.

VALIDAÇÃO PRÁTICA

Testes de Validação e Persistência

Entradas Inválidas

Testadas notas com letras e valores fora do intervalo (ex: -5 e 12).

Nomes Duplicados

Validação impede o cadastro repetido de estudantes já existentes.

Reinício da Aplicação

Confirmada a leitura completa do arquivo `boletim.txt` ao reabrir.

Conclusão e Aprendizados

O projeto consolidou a aplicação prática de estruturas de dados, modularização em funções e gerenciamento de arquivos em Python para resolver um problema real.

Obrigado! Perguntas ou Considerações?

Image Sources



https://cdn.prod.website-files.com/689f009387b6b6b947c8a835/6a85ef3b6540b886b49b17e7_6a85ef36d740698fde2dc616_6a846b8ca909cf0505b1fb81-6a846b898d731e1a158b306b-6a5600f415.webp

Source: www.immerse.education